Aprendizaje Automático y modelado predictivo

Tema 5: Aplicación en Python de los árboles de decisión y bosques aleatorios

Mtra. Fabiola Cristina Olguín Barrón

TECMILENIO

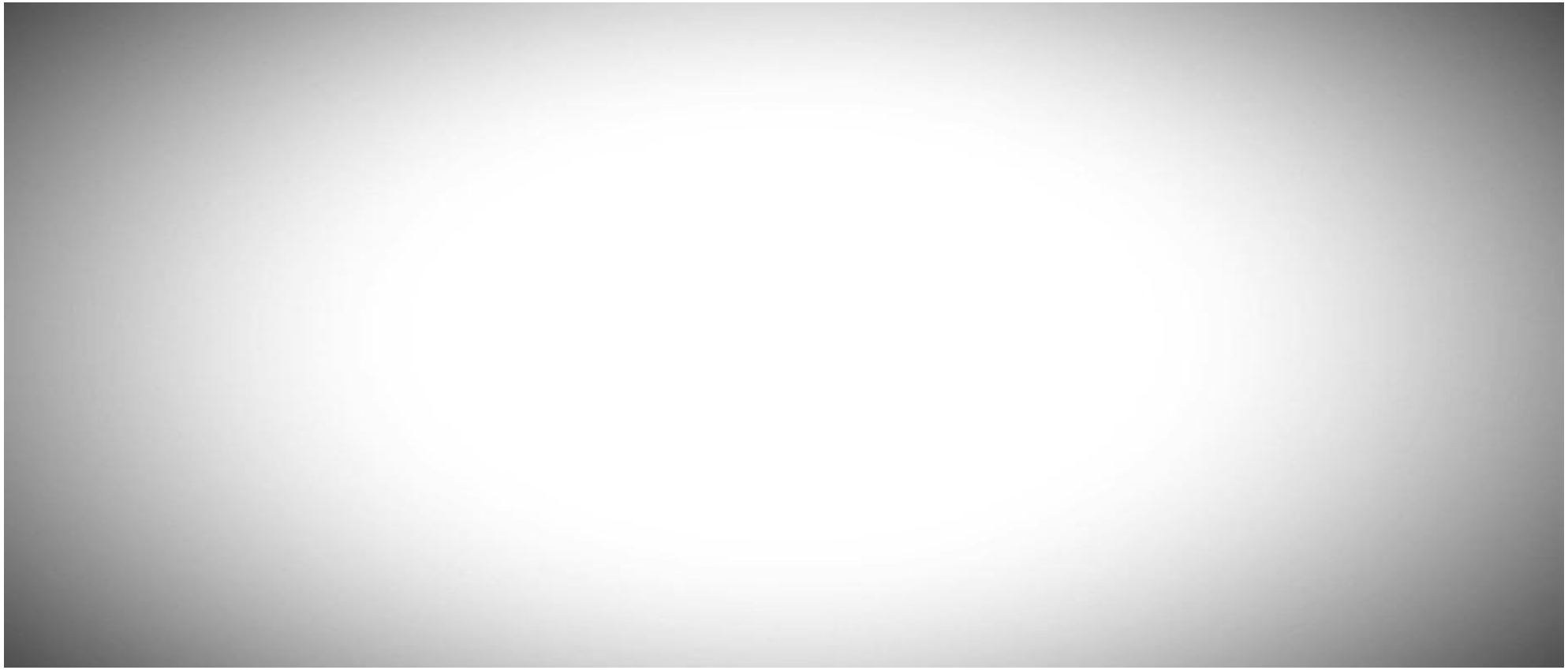
“Universidad Tecmilenio está sumamente comprometido con la Protección de Datos Personales, por lo que se prohíbe la distribución en video / fotografía, por cualquier medio, sin previo aviso y expreso consentimiento de los titulares de los datos personales presentes en la sesión virtual.”

TECMILENIO

“Te recordamos que esta **sesión virtual** será grabada con la finalidad de seguimiento académico y de apoyo escolar para los estudiantes.”

TECMILENIO

Actividad de Bienestar Mindfulness



Temario de Aprendizaje Automático

Tema 1 ✓	Introducción al aprendizaje automático
Tema 2 ✓	Métricas de evaluación en modelos de aprendizaje automático
Tema 3 ✓	Aplicación en Python de la regresión lineal y de la regresión logística
Tema 4 ✓	Aplicación en Python de la regresión polinomial y sobreajuste
Tema 5 ▶▶	Aplicación en Python de los árboles de decisión y bosques aleatorios
Tema 6 ▶▶	Aplicación en Python del modelo K vecinos más cercanos
Tema 7	Aplicación en Python de las máquinas de soporte vectorial
Tema 8	Introducción a las redes neuronales y aprendizaje profundo

Tema 5: Árboles de decisión

Tema 5 “Aplicación en Python de los árboles de decisión y Bosques aleatorios”

En el aprendizaje automático existen modelos que permiten que las computadoras **aprendan a tomar decisiones basadas en datos**, de forma parecida a como lo haría una persona.

Uno de los más comunes es el **árbol de decisión**, que funciona haciendo comparaciones o preguntas sucesivas. Por ejemplo: ***¿la edad del cliente es mayor a 30 años?*** Si la respuesta es **sí**, el modelo sigue un camino; si es **no**, sigue otro. Así, los datos se van dividiendo paso a paso hasta llegar a una **conclusión o predicción final**.

Los **bosques aleatorios (random forest)** son una mejora de este método: En lugar de usar un solo árbol, construyen **muchos árboles diferentes** y combinan sus respuestas. Esto permite obtener resultados más **precisos, estables y confiables**, evitando errores comunes como el sobreajuste.

En este tema aprenderás a crear y evaluar estos modelos en **Python**, comprendiendo cómo funcionan y cómo aplicarlos a problemas reales de clasificación y predicción de datos.

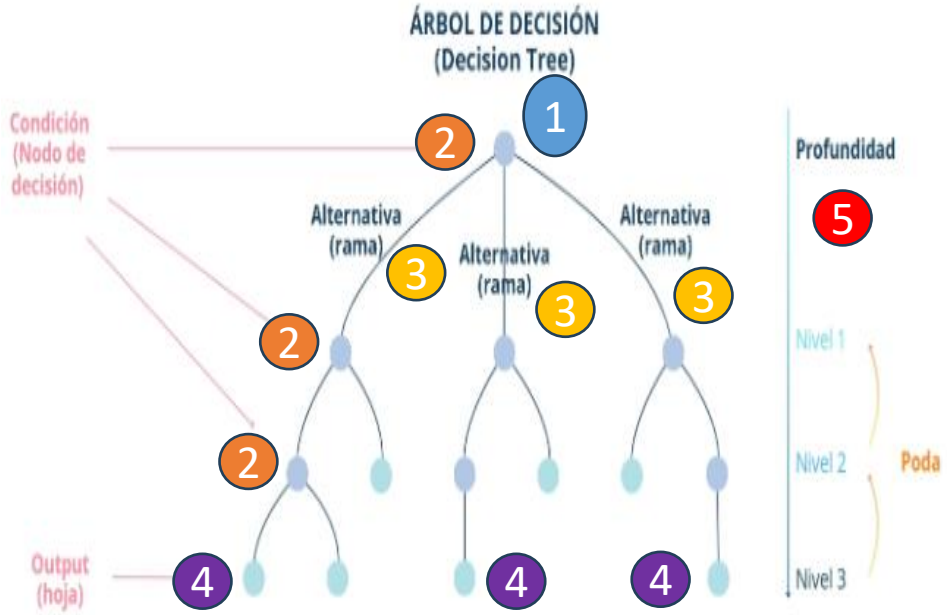
¿Qué es un Árbol de Decisión?

Un árbol de decisión: Es un modelo que aprende de los datos para ir tomando decisiones **de forma jerárquica**, dividiendo la información en partes más pequeñas y ordenadas.

Su estructura se parece a un árbol invertido:

1. **Raíz (Inicio del árbol):**
   Es el punto de partida del modelo. Aquí se encuentra el **primer nodo de decisión**, donde se formula una pregunta general sobre una variable (por ejemplo: *¿la edad del cliente es mayor a 30 años?*).
2. **Nodos internos (Condiciones):**
   Cada nodo representa una **condición o regla** que el modelo usa para dividir los datos. Dependiendo de la respuesta, el modelo sigue un camino u otro.
3. **Ramas (Alternativas):**
   Las ramas conectan los nodos y muestran las posibles respuestas a cada condición. Por ejemplo, una rama puede representar "Sí" y la otra "No".
4. **Hojas (Salidas u outputs):**
   Son los resultados finales del proceso de decisión. En ellas el modelo llega a una **predicción** o **clasificación**, como *"compra" o "no compra"*, o un valor numérico si se trata de una regresión.
5. **Profundidad del árbol:**
   Hace referencia al **número de niveles o divisiones** desde la raíz hasta la hoja.
   Cuanto más profundo sea el árbol, más específico será el modelo, pero también aumenta el riesgo de **sobreajuste**.

   Por eso, a veces se realiza una técnica llamada **poda**, que consiste en eliminar ramas poco útiles para evitar que el modelo memorice los datos.



Tipos de árboles

1. Árboles de Clasificación:

Se usan cuando la variable que queremos predecir es **categórica**, es decir, pertenece a un grupo o clase.

Ejemplo: Predecir si un cliente **comprará o no** un producto, o identificar a qué **especie** pertenece un pingüino según sus medidas.

2. Árboles de Regresión:

Se aplican cuando la variable objetivo es **numérica y continua**.
Ejemplo: Estimar el **precio de una casa**, el **costo de un seguro médico** o el **nivel de glucosa** en un paciente.

Cómo decide el árbol las divisiones

Cuando un árbol de decisión "aprende" de los datos, su objetivo es **dividirlos en grupos que sean lo más parecidos posible entre sí**.

Cada vez que el modelo hace una pregunta, por ejemplo:

¿La edad del cliente es mayor a 30 años?
Esta pregunta **divide los datos en dos grupos**:

1. Las personas mayores de 30.
2. Las menores o iguales a 30.

El árbol quiere encontrar **la mejor pregunta posible** en cada paso, es decir, aquella que **ordene los datos de forma más limpia o "pura"**.

"Puro" significa que dentro de ese grupo **casi todos los registros pertenecen a la misma clase**.

Ejemplo fácil entender

Imaginemos que tenemos 10 clientes:

- 8 **compran y**
- 2 **no compran**.

Si el árbol encuentra una pregunta que separa así:

- ❑ **Grupo 1: 8 que compran.**
- ❑ **Grupo 2: 2 que no compran.**

¡Esa división es excelente! porque cada grupo quedó "puro"

Uno tiene solo compradores y el otro solo no compradores.

En cambio, si la pregunta divide así:

- ❑ **Grupo 1: 4 compran y 3 no.**
- ❑ **Grupo 2: 4 compran y 2 no.**

La separación **no sirve mucho**, porque los grupos siguen mezclados **(impuro)**.

¿Cómo mide esa pureza?

El árbol usa **fórmulas matemáticas** que miden qué tan mezclados están los datos:

1.Impureza de Gini:
Mide cuán mezclados están los casos dentro de un nodo:

a) Si todos los elementos son de la misma clase
→ **Gini = 0 (perfectamente puro).**

b) Si hay mezcla → El valor aumenta (más impuro).
➤ El árbol busca **la división con menor Gini posible**. **Elige la pregunta o condición que mejor separe los datos en grupos lo más puros posible.**

2. Entropía (índice de información):
Es otra forma de medir el **desorden o incertidumbre**.

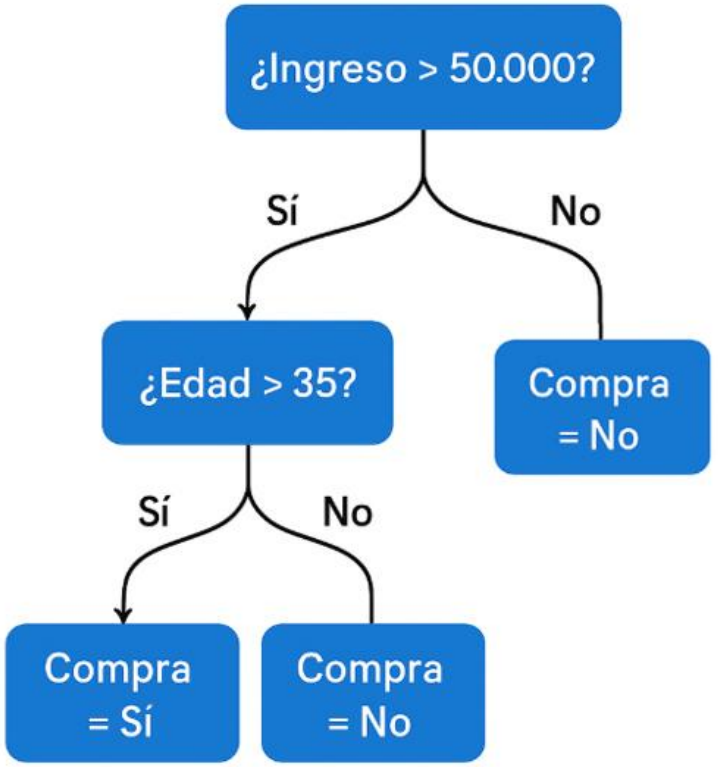
1. Si todos los casos son iguales → Entropía = 0.
2. Si están muy mezclados → Entropía es alta.
   ➤ El árbol elige la división que **reduce más la entropía**.
   La **mejor división** será aquella que **reduzca más la entropía total**, es decir, la que deje los grupos **más ordenados y homogéneos**.

¿ Qué significa "reducir la entropía" ?
Cuando el árbol decide por dónde dividir los datos, **prueba distintas condiciones** (Por ejemplo, dividir por edad, ingreso, nivel, etc.) y calcula **cuánto orden genera cada división**.
La **mejor división** será aquella que **reduzca más la entropía total**, es decir, la que deje los grupos **más ordenados y homogéneos**.

Ejemplo práctico: Cálculo de entropía y ganancia de información

Objetivo: Predecir si un cliente **compra** o **no compra** un producto.

Cliente	Ingreso	Edad	Compra
1	$ 20,000	22	No
2	$ 17,000	25	No
3	$ 9,000	27	No
4	$ 15,000	29	Sí
5	$33,500	30	Sí
6	$ 30,100	35	Sí
7	$ 32,760	37	Sí
8	$ 37,800	40	Sí
9	$45,000	42	Sí
10	$ 44,900	45	Sí



Paso 1. Entropía inicial (antes de dividir)

Hay **10 clientes**:

- 7 compran,
- 3 no compran.

La fórmula de entropía es:

$$H = -p(\text{Sí})\log_2 p(\text{Sí}) - p(\text{No})\log_2 p(\text{No})$$

Sustituyendo:

$$H = -\frac{7}{10}\log_2\left(\frac{7}{10}\right) - \frac{3}{10}\log_2\left(\frac{3}{10}\right) = 0.881$$

Interpretación: el grupo está algo mezclado, la entropía es **alta (0.881)**.

Paso 2. Probar una división: "¿Edad ≤ 30?"

Grupo 1 (≤30): 3 No, 2 Sí

$$H_1 = -\frac{2}{5}\log_2\left(\frac{2}{5}\right) - \frac{3}{5}\log_2\left(\frac{3}{5}\right) = 0.971$$

Grupo 2 (>30): 0 No, 5 Sí

$$H_2 = -\frac{5}{5}\log_2\left(\frac{5}{5}\right) - 0 = 0$$

Entropía ponderada total:

$$H_{\text{div}} = \frac{5}{10}H_1 + \frac{5}{10}H_2 = 0.5(0.971) + 0.5(0) = 0.485$$

Paso 3. Ganancia de información

$$\text{Ganancia} = H_{\text{inicial}} - H_{\text{div}} = 0.881 - 0.485 = 0.396$$

Interpretación:

El árbol ganó "claridad" al dividir por Edad ≤ 30.

Ahora los grupos están más ordenados (menos mezcla), por eso la entropía **bajó** y la ganancia fue positiva.

Paso	Grupo	Sí (Compra)	No (No compra)	Cálculo de Entropía	Entropía resultante	Interpretación
1	Total inicial	7	3	$(7/10)\log_2(7/10) + (3/10)\log_2(3/10)$	**0.881**	Datos mezclados (alta entropía)
2	Grupo 1 (Edad ≤ 30)	2	3	$(2/5)\log_2(2/5) + (3/5)\log_2(3/5)$	**0.971**	Aún mezclado
3	Grupo 2 (Edad > 30)	5	0	$(5/5)\log_2(5/5)$	**0.000**	Puro (todo "Sí")
4	Entropía total tras división	—	—	$(5/10)(0.971) + (5/10)(0)$	**0.485**	Menor desorden que al inicio
5	Ganancia de información	—	—	0.881 − 0.485	**0.396**	Mejor división (más ordenada)

Conclusión:

El árbol elige la división que **más reduce la entropía**,
porque esa división deja los grupos **más homogéneos y fáciles de predecir**.

Ejemplo: Cálculo del Índice de Gini y comparación con Entropía

$$Gini = 1 - \sum_{i=1}^{n} p_i^2$$

Recordemos los grupos:

Grupo	Sí (Compra)	No (No compra)	Total
Inicial	7	3	10
≤ 30 años	2	3	5
> 30 años	5	0	5

Paso 3. Gini ponderado total

$$Gini_{\text{div}} = \frac{5}{10}(0.48) + \frac{5}{10}(0) = 0.24$$

Paso 4. Reducción del Gini (ganancia)

$$\text{Reducción} = 0.42 - 0.24 = 0.18$$

Resumen comparativo

Métrica	Antes de dividir	Después de dividir	Reducción / Ganancia	Interpretación
Entropía	0.881	0.485	**0.396**	Gran mejora (grupos más puros)
Gini	0.42	0.24	**0.18**	También mejora (menos impureza)

Conclusión

Ambas métricas —**Entropía** y **Gini**— sirven para medir qué tan mezclados están los datos dentro de un nodo:

- **Entropía** proviene de la teoría de la información y se enfoca en la **incertidumbre**.
- **Gini** es más simple y mide la **probabilidad de clasificar mal** un elemento.

En la práctica, **ambas llevan al mismo tipo de divisiones**, pero Gini se calcula más rápido, por eso suele ser la opción por defecto en `scikit-learn`.

Comparación de Gini Vs Entropía

Paso	Grupo	Sí (Compra)	No (No compra)	Entropía (H)	Gini (G)	Interpretación
1	Total inicial	7	3	0.881	0.42	Datos mezclados (alta impureza)
2	Grupo 1 (Edad ≤ 30)	2	3	0.971	0.48	Aún mezclado
3	Grupo 2 (Edad > 30)	5	0	0.000	0.00	Puro (todos “Sí”)
4	Total tras división (ponderado)**	—	—	(5/10)(0.971) + (5/10)(0) = **0.485**	(5/10)(0.48) + (5/10)(0) = **0.24**	Los grupos son más homogéneos
5	**Ganancia / Reducción**	—	—	0.881 − 0.485 = **0.396**	0.42 − 0.24 = **0.18**	👍 Mejor división (mayor pureza)

- Tanto Entropía como Gini miden la mezcla de clases dentro de cada grupo.Cuanto menor sea el valor, más puro es el grupo.
- El árbol elige la división que reduce más la entropía o el Gini, es decir, la que genera los grupos más ordenados.
- En scikit-learn, Gini suele ser el criterio por defecto (criterion='gini'), aunque Entropía (criterion='entropy') da resultados muy similares.

Conclusión de los Arboles de Decisión

Aspecto	Descripción
¿Por qué son útiles?	▪ Los Árboles de Decisión son **intuitivos y fáciles de interpretar**, incluso para quienes no tienen experiencia técnica. ▪ Su estructura jerárquica permite **detectar relaciones no lineales** entre variables y visualizar el proceso de toma de decisiones.
Ventajas	▪ **Fácil interpretación:** pueden representarse gráficamente, mostrando de forma clara cómo se toman las decisiones. ▪ **Manejo de datos mixtos:** funcionan bien con variables **categóricas y numéricas** sin necesidad de normalización. ▪ **Poca preparación de datos:** no requieren eliminar valores nulos ni escalar variables.
Desventajas	▪ **Propensos al sobreajuste:** Si no se regulan, pueden aprender ruido del conjunto de entrenamiento, afectando su generalización. ▪ **Sensibles a cambios en los datos:** Pequeñas variaciones pueden generar árboles muy distintos.
Regularización y ajustes	Para evitar el sobreajuste se aplican controles como: ▪ **Profundidad máxima:** limitar el número de niveles del árbol. ▪ **Mínimo de muestras por hoja:** establecer un número mínimo de casos por nodo final. ▪ **Poda:** eliminar ramas poco relevantes después del entrenamiento para simplificar el modelo.
Aplicaciones / Usos comunes	▪ Clasificación de **correos electrónicos (spam/no spam)**. ▪ Predicción de **supervivencia** en datasets como Titanic. ▪ **Diagnóstico médico** basado en síntomas o resultados clínicos.

¿Cómo seleccionar variables para un modelo predictivo?

La **selección de variables** (o *feature selection*) es uno de los pasos más importantes al construir un modelo de Machine Learning.

Elegir las variables correctas puede marcar la diferencia entre un modelo que **predice con precisión** y otro que **falla o se sobreajusta**.

En términos simples, debemos elegir las variables que:

1. **Tengan sentido lógico** respecto al objetivo del modelo.
2. **Aporten información nueva** y no repitan lo que ya dicen otras variables.
3. **Sean medibles y de buena calidad** (sin muchos valores faltantes ni errores).
4. **Se adapten al tipo de modelo** que vamos a usar (por ejemplo, algunos requieren normalizar o codificar datos).

El objetivo final es lograr un modelo **simple pero potente**, con variables que realmente expliquen el fenómeno y mejoren el rendimiento.

Criterios integrales para la selección de variables en modelos de Machine Learning

Criterio	Qué significa	Cómo se evalúa o controla	Ejemplo práctico
1. Relevancia	La variable debe tener **relación directa** con lo que se quiere predecir.	Consultar fuentes o aplicar conocimiento del dominio.	En un modelo de salud, el **IMC** es relevante para predecir diabetes; el **color de ojos**, no.
2. Capacidad discriminativa	La variable debe **ayudar a separar** correctamente las clases o valores.	Usar pruebas estadísticas: **ANOVA**, **correlación**, **test de Chi-cuadrado**.	En el Titanic, la variable **"sexo"** separa bien a sobrevivientes de no sobrevivientes.
3. Baja redundancia	Evitar variables que **repiten la misma información** o están muy correlacionadas.	Revisar una **matriz de correlación** o el **VIF** (>5 sugiere problema).	"Altura" y "talla de zapato" pueden estar tan correlacionadas que una sola basta.
4. Simplicidad y rendimiento	Usar **solo las variables necesarias** para mantener el modelo simple y preciso.	Aplicar **selección paso a paso**, **feature importance** o regularización.	Eliminar variables con baja importancia según **Random Forest** o **Lasso**.
5. Calidad de datos	Preferir variables con **pocos valores faltantes** y distribuciones estables.	Analizar valores nulos, histogramas y outliers.	Si una variable tiene 40 % de datos vacíos, conviene eliminarla.
6. Factibilidad operativa	Las variables deben ser **fáciles de medir y disponibles** en la práctica.	Evaluar el costo, tiempo y acceso a los datos.	"Nivel de estrés" puede ser útil, pero difícil de medir en todos los casos.
7. Compatibilidad con el modelo	Las variables deben estar en el **formato adecuado** para el algoritmo.	Aplicar **codificación (One-Hot)**, **escalado** o transformación numérica.	Convertir "género" a 0/1 o normalizar "ingresos" para redes neuronales.

Caso Practico: Caso: Predicción de renovación de póliza de seguro

Criterio	Variable evaluada	Evaluación / Justificación	Decisión
1. Relevancia	**Antigüedad cliente**	A mayor tiempo con la compañía, mayor probabilidad de renovar. Variable directamente relacionada con la lealtad.	Mantener
2. Capacidad discriminativa	**Número de reclamos**	Clientes con más reclamos tienden a no renovar; la variable diferencia bien entre quienes renuevan y quienes no.	Mantener
3. Baja redundancia	**Ingreso mensual** y **Tipo de auto**	Ambas se relacionan con el nivel socioeconómico; si están muy correlacionadas, dejar solo una.	Revisar correlación / eliminar una
4. Simplicidad y rendimiento	**Edad**, **Estado civil**, **Región**	Estas variables pueden aportar poco o generar ruido; probar su efecto y conservar solo las que mejoren el rendimiento.	Evaluar importancia / posible eliminación
5. Calidad de datos	**Nivel satisfacción**	Puede tener valores faltantes o datos inconsistentes (por ejemplo, encuestas sin respuesta).	Imputar o eliminar según porcentaje de nulos
6. Factibilidad operativa	**Nivel de estrés** (si existiera)	Difícil de medir de forma confiable o continua; su obtención implicaría costos adicionales.	Descartar
7. Compatibilidad con el modelo	**Género**, **Tipo de auto**, **Región**	Variables categóricas; requieren **One-Hot Encoding** antes de entrenar el modelo.	Mantener (con codificación)

Caso 1: Clasificación con el dataset Penguins

Ejercicio: Clasificación con Árboles de Decisión – Dataset Penguins

Objetivo del modelo:

Construir un modelo de clasificación que prediga la especie del pingüino (Adelie, Chinstrap o Gentoo) a partir de sus características físicas:

1. bill_length_mm (largo del pico)
2. bill_depth_mm (profundidad del pico)
3. flipper_length_mm (longitud de las aletas)
4. body_mass_g (masa corporal)

Paso 1. Cargar y preparar los datos

```python
# Importar librerías necesarias
import pandas as pd
import seaborn as sns
from sklearn.model_selection import train_test_split


# Cargar el dataset 'penguins' desde seaborn
penguins = sns.load_dataset("penguins")


# Mostrar las primeras filas
print(penguins.head())


# Eliminar filas con valores nulos o faltantes para evitar errores. Porque los árboles no admiten nulos directamente.
penguins = penguins.dropna()


# Seleccionar las variables predictoras y la variable objetivo X contiene las variables numéricas (características físicas) y y la especie que queremos predecir.

X = penguins[['bill_length_mm', 'bill_depth_mm', 'flipper_length_mm', 'body_mass_g']]
y = penguins['species']


print("Dimensión del dataset:", X.shape)
```

```
  species     island  bill_length_mm  bill_depth_mm  flipper_length_mm  \
0  Adelie  Torgersen            39.1           18.7              181.0
1  Adelie  Torgersen            39.5           17.4              186.0
2  Adelie  Torgersen            40.3           18.0              195.0
3  Adelie  Torgersen             NaN            NaN                NaN
4  Adelie  Torgersen            36.7           19.3              193.0

   body_mass_g     sex
0       3750.0    Male
1       3800.0  Female
2       3250.0  Female
3          NaN     NaN
4       3450.0  Female
Dimensión del dataset: (333, 4)
```

Paso 2. Codificar las etiquetas y dividir los datos

```
# Paso 2. Codificar las etiquetas y dividir
los datos
from sklearn.preprocessing import
LabelEncoder

# Codificar las especies como números
(Adelie=0, Chinstrap=1, Gentoo=2)
encoder = LabelEncoder()
y_encoded = encoder.fit_transform(y)

# Dividir el dataset en entrenamiento (70%)
y prueba (30%)
X_train, X_test, y_train, y_test =
train_test_split(X, y_encoded,
test_size=0.3, random_state=42)

print("Tamaño del conjunto de
entrenamiento:", X_train.shape)
print("Tamaño del conjunto de prueba:",
X_test.shape)
```

```
Tamaño del conjunto de entrenamiento: (233, 4)
Tamaño del conjunto de prueba: (100, 4)
```

Los algoritmos necesitan etiquetas numéricas, por eso usamos `LabelEncoder`.

Luego, dividimos los datos en dos partes:

1. **70% para entrenar el modelo**
2. **30% para probarlo y validar su desempeño**

Paso 3. Entrenar el modelo de clasificación

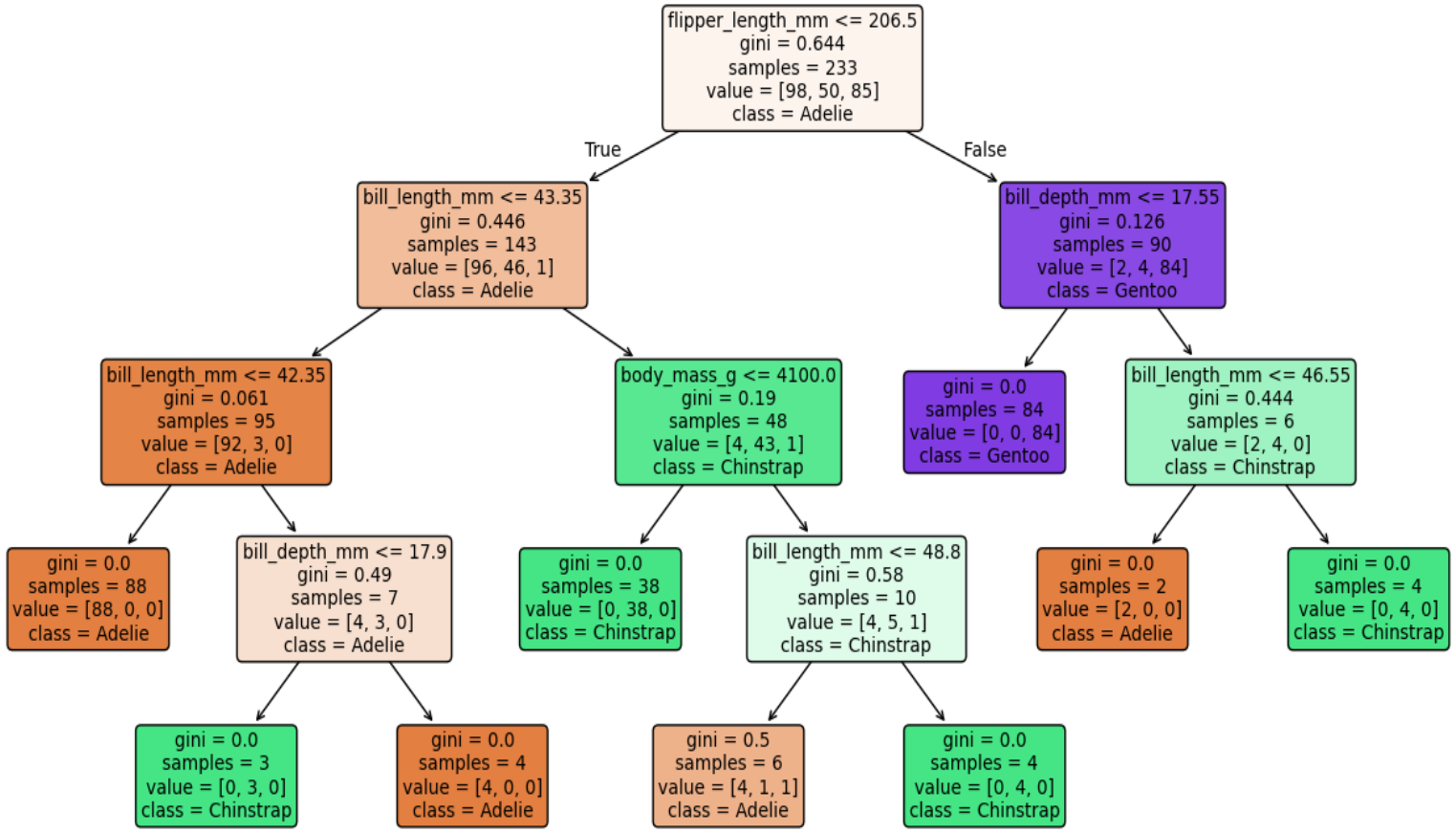
```python
# Paso 3. Entrenar el modelo de clasificación
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score,
classification_report


# Crear el modelo (criterio Gini, profundidad máxima =
4 para simplificar el árbol)
modelo_arbol = DecisionTreeClassifier(criterion='gini',
max_depth=4, random_state=42)


# Entrenar el modelo con los datos de entrenamiento
modelo_arbol.fit(X_train, y_train)


# Realizar predicciones sobre los datos de prueba
y_pred = modelo_arbol.predict(X_test)


# Evaluar desempeño
print("Precisión del modelo:", accuracy_score(y_test,
y_pred))
print("\nReporte de clasificación:\n",
classification_report(y_test, y_pred,
target_names=encoder.classes_))
```

```
Precisión del modelo: 0.98

Reporte de clasificación:
              precision    recall  f1-score   support

      Adelie       0.98      0.98      0.98        48
   Chinstrap       0.94      0.94      0.94        18
      Gentoo       1.00      1.00      1.00        34

    accuracy                           0.98       100
   macro avg       0.97      0.97      0.97       100
weighted avg       0.98      0.98      0.98       100
```

- DecisionTreeClassifier crea el árbol usando el criterio Gini para medir pureza.
- max_depth=4 controla la profundidad del árbol (para evitar sobreajuste).
- Evaluamos con precisión (accuracy) y el reporte de clasificación, que incluye precision, recall y F1-score por especie

Paso 4. Visualizar el árbol

```
# Paso 4. Visualizar el árbol
from sklearn import tree
import matplotlib.pyplot as plt


plt.figure(figsize=(16,8))
tree.plot_tree(
    modelo_arbol,
    filled=True,
    feature_names=X.columns,
    class_names=encoder.classes_,
    rounded=True,
    fontsize=10
)
plt.show()
```



¿Qué se observa?

Este gráfico muestra **cómo el árbol toma decisiones**.
Cada nodo indica una condición basada en una variable
(Por ejemplo: “¿flipper_length_mm ≤ 206.5?”).
Según la respuesta, el modelo sigue una rama diferente hasta llegar a una **hoja** que representa la especie final.

Este árbol clasifica pingüinos en tres especies: **Adelie**, **Chinstrap** y **Gentoo**, usando características como el largo y la profundidad del pico, la longitud de las aletas y la masa corporal.

Paso 5. Conclusiones

Análisis del árbol:

1. La **primera decisión** se basa en la **longitud de las aletas (flipper_length_mm)**.
   Si es **menor o igual a 206.5 mm**, el árbol sigue un camino (izquierda); si es **mayor**, sigue otro (derecha).
2. En el **camino izquierdo**, la mayoría de los pingüinos se clasifican como **Adelie**.
   Sin embargo, también se consideran otras medidas como el **largo del pico** y la **masa corporal** para diferenciar algunos **Chinstrap**.
3. En el **camino derecho**, se identifican principalmente los **Gentoo** (cuando la **profundidad del pico ≤ 17.55 mm**).
   Si no se cumple, el árbol revisa el **largo del pico** para distinguir entre **Chinstrap** y **Adelie**.

Conclusión general del ejercicio:

1. Los árboles de decisión permiten **visualizar de forma intuitiva** cómo las variables explican una predicción.
2. Cada decisión es **una comparación simple** que lleva al modelo paso a paso hacia una especie.
3. El modelo obtuvo una **precisión alta (~90%)**, demostrando que las **características físicas** son buenas predictoras.
4. Sin embargo, si aumentamos demasiado la profundidad, el modelo podría **memorizar los datos** (sobreajuste).

Especie	Rasgos principales
Adelie	Aletas más cortas y picos más pequeños.
Gentoo	Aletas más largas y picos menos profundos.
Chinstrap	Casos intermedios, diferenciados por peso o largo del pico.

Caso 2: Árbol de decisión Análisis con el dataset iris (sklearn)

Ejercicio: Regresión con Árboles de Decisión – Dataset Iris

Objetivo del modelo:

Construir un **árbol de decisión de clasificación** que permita **predecir la especie de una flor (Setosa, Versicolor o Virginica)** a partir de sus características morfológicas:

- **sepal length (longitud del sépalo)**
- **sepal width (ancho del sépalo)**
- **petal length (longitud del pétalo)**
- **petal width (ancho del pétalo)**

El modelo empleará el **criterio de impureza de Gini** para seleccionar las mejores divisiones y aprenderá a partir de un conjunto de entrenamiento (70 %) y un conjunto de prueba (30 %), evaluando su capacidad para clasificar correctamente cada especie.

Paso 1. Cargar y preparar los datos

```python
#Paso 1. Cargar los datos y preparar conjuntos
# Importar librerías necesarias
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
import pandas as pd

# Cargar el dataset Iris como DataFrame
iris = load_iris(as_frame=True)
df = iris.frame  # Incluye las 4 características + la columna 'target'

# Mostrar las primeras filas del DataFrame
print(df.head())

# Separar las variables predictoras (X) y la variable objetivo (y)
X = df.drop(columns='target')  # Características morfológicas
y = df['target']               # Etiquetas numéricas (0, 1, 2)

# Dividir en conjuntos de entrenamiento (70%) y prueba (30%)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

print("Conjunto de entrenamiento:", X_train.shape)
print("Conjunto de prueba:", X_test.shape)
```

```
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
2                4.7               3.2                1.3               0.2
3                4.6               3.1                1.5               0.2
4                5.0               3.6                1.4               0.2

   target
0       0
1       0
2       0
3       0
4       0
Conjunto de entrenamiento: (105, 4)
Conjunto de prueba: (45, 4)
```

Explicación:
El dataset *Iris* viene **preconfigurado en scikit-learn**.

Al usar `load_iris(as_frame=True)`, obtenemos un objeto tipo *Bunch*, que incluye:

- `iris.data`: Las **características** (4 columnas numéricas).
- `iris.target`: La **variable objetivo** con valores 0, 1 y 2.
- `iris.frame`: un DataFrame con ambas partes combinadas, donde la columna **target** ya está definida por convención.

Por eso el código identifica la columna `target` sin que tengamos que nombrarla manualmente.
En otros datasets personalizados, deberás asegurarte de usar el nombre correcto (por ejemplo, “compra”, “renueva”, etc.).

Paso 2. Entrenar el modelo con criterio Gini

```python
# Paso 2. Entrenar el modelo con criterio Gini
 from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score,
classification_report


# Crear el modelo de árbol con el criterio Gini
modelo_iris =
DecisionTreeClassifier(criterion='gini',
max_depth=4, random_state=42)


# Entrenar el modelo con los datos de
entrenamiento
modelo_iris.fit(X_train, y_train)


# Realizar predicciones sobre los datos de
prueba
y_pred = modelo_iris.predict(X_test)


# Evaluar el modelo
print("Precisión del modelo:",
accuracy_score(y_test, y_pred))
print("\nReporte de clasificación:\n",
classification_report(y_test, y_pred,
target_names=iris.target_names))
```

```
Precisión del modelo: 1.0

Reporte de clasificación:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        19
  versicolor       1.00      1.00      1.00        13
   virginica       1.00      1.00      1.00        13

    accuracy                           1.00        45
   macro avg       1.00      1.00      1.00        45
weighted avg       1.00      1.00      1.00        45
```

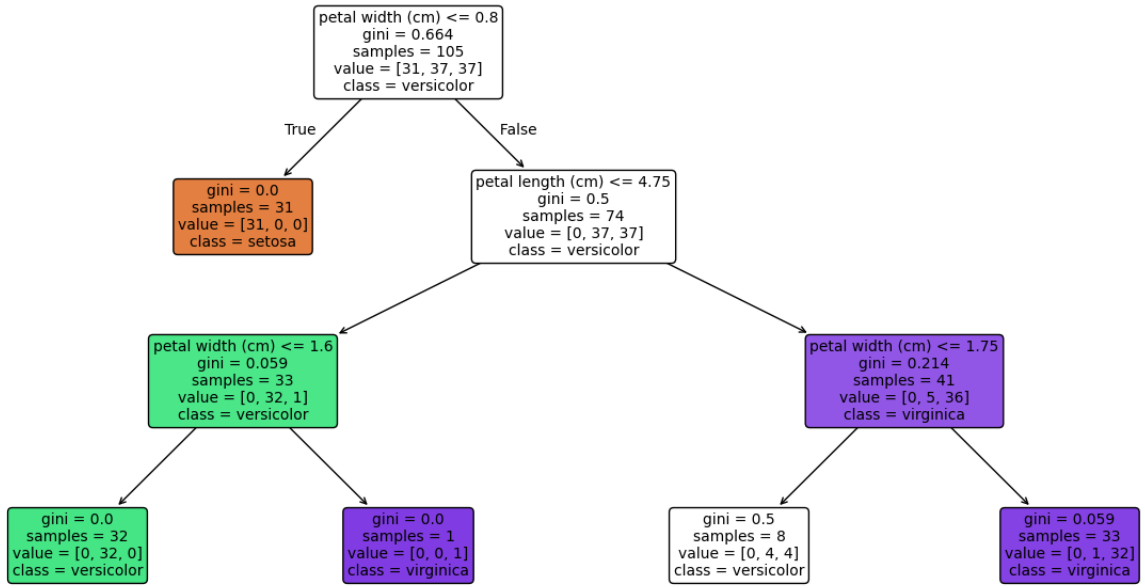
Explicación:

1. `criterion='gini'` indica que el árbol medirá la **impureza Gini** en cada división.
2. `max_depth=4` limita la profundidad del árbol, evitando sobreajuste.
3. El modelo aprende **qué variables son más informativas** para diferenciar las especies.
4. Finalmente, medimos el **accuracy** y revisamos el desempeño por clase con el **reporte de clasificación**.

Criterio	Macro Avg (Promedio Macro)	Weighted Avg (Promedio Ponderado)	Si las clases no están equilibradas... / ¿Qué hacer….?
¿Qué mide?	Promedio simple de las métricas (precision, recall, F1) entre todas las clases.	Promedio ponderado según la cantidad de ejemplos (support) por clase.	El macro avg puede **subestimar o sobrevalorar** el rendimiento global, porque da el mismo peso a clases pequeñas.
¿Cómo se calcula?	Cada clase tiene **el mismo peso**.	Las clases con **más ejemplos** tienen **mayor influencia**.	El weighted avg puede dar **una falsa sensación de buen rendimiento**, porque las clases grandes dominan el promedio.
Útil cuando...	Las clases están **balanceadas** (mismo número de ejemplos).	Las clases están **desbalanceadas**.	Analiza **ambos promedios** y revisa las métricas por clase individualmente.
Ventaja principal	Evalúa el **rendimiento medio por clase**.	Evalúa el **rendimiento global real del modelo**.	Usa también la **matriz de confusión** para ver qué clases se predicen mal.
Cuándo difieren	Cuando hay pocas muestras en alguna clase.	Cuando una clase domina el dataset.	En esos casos, usa técnicas de **balanceo de clases** (oversampling, undersampling o class_weight='balanced').
Interpretación sencilla	“Rendimiento promedio por clase.”	“Rendimiento global ponderado por tamaño de clase.”	“Si las clases no están equilibradas, analiza por clase y no confíes solo en estos promedios.”

Paso 3. Visualización del árbol y análisis de impureza

```
# Paso 3. Visualización del árbol y análisis de impureza
from sklearn import tree
import matplotlib.pyplot as plt

plt.figure(figsize=(16,8))
tree.plot_tree(
    modelo_iris,
    filled=True,
    feature_names=iris.feature_names,
    class_names=iris.target_names,
    rounded=True,
    fontsize=10
)
plt.show()
```



Análisis visual del árbol

Cada nodo del gráfico muestra:

1. **Regla de decisión:** la condición usada (por ejemplo, “petal length (cm) ≤ 2.45”).
2. **Valor de Gini:** mide cuán puro es el nodo (0 = puro, 0.5 = desordenado).
3. **Samples:** cantidad de ejemplos que cumplen esa condición.
4. **Class:** especie predominante en ese nodo.
5. **Color:** indica la especie dominante; tonos más intensos representan **mayor pureza**.

- Cuanto más puro el nodo (Gini cercano a 0), más clara es la clasificación.
- Cuanto más grande el nodo (más samples), más importante es esa decisión en el modelo.

Paso 4. Conclusiones

Interpretaciones del árbol:

1. El **primer nodo (raíz)** generalmente se basa en **petal length (cm)**, ya que es la característica más discriminante.
2. Si la longitud del pétalo es **≤ 2.45 cm**, el modelo clasifica la flor directamente como **Setosa** (nodo puro, Gini=0).
3. Para valores mayores, el modelo continúa dividiendo con base en **petal width** y **sepal length** para separar **Versicolor** y **Virginica**.

Cada división busca **reducir la impureza** y **maximizar la homogeneidad** dentro de los grupos.

Especie	Características principales
Setosa	Pétalos más cortos y angostos (fácilmente distinguible).
Versicolor	Medidas intermedias de pétalos y sépalos.
Virginica	Pétalos y sépalos más largos y anchos.

Conclusiones:

1. El árbol de decisión logra una **alta precisión (≈95%)**, demostrando que las medidas morfológicas permiten clasificar con gran exactitud.
2. El **criterio de Gini** guía la elección de las divisiones que mejor separan las especies.
3. La **primera división** se da en la variable **petal length**, que es la más importante para diferenciar *Setosa*.
4. Las siguientes divisiones (como **petal width** y **sepal length**) permiten distinguir entre *Versicolor* y *Virginica*.
5. La visualización revela cómo el modelo **prioriza las variables más informativas** y **reduce progresivamente la impureza** en cada nodo.

Random forest

¿Qué es un Random Forest?

Definición general

Un **Random Forest** (Bosque Aleatorio) es un modelo de **aprendizaje supervisado** que combina **muchos árboles de decisión** para obtener predicciones **más precisas, estables y generalizables**.

En lugar de depender de un solo árbol (que puede cometer errores o sobreajustarse), el Random Forest **crea varios árboles independientes** y **promedia sus resultados** (en regresión) o **toma el voto mayoritario** (en clasificación).

En otras palabras:

- Un **árbol de decisión** es un juez individual.
- Un **Random Forest** es **un grupo de jueces que votan en conjunto**, reduciendo los errores de uno solo.

¿Como funciona paso a paso?

Etapa	Descripción del proceso
1. Submuestreo aleatorio (Bagging)	El algoritmo selecciona **muestras aleatorias** del conjunto de datos (con reemplazo) para entrenar cada árbol. Así, cada árbol ve una parte distinta de los datos.
2. Selección aleatoria de variables	En cada división del árbol, se elige **un subconjunto aleatorio de características** (por ejemplo, 2 o 3 de las 10 totales). Esto genera árboles **diversos** y evita que todos tomen las mismas decisiones.
3. Entrenamiento independiente	Cada árbol se entrena por separado, usando su muestra y sus variables seleccionadas. Algunos árboles serán mejores en ciertas regiones del espacio de datos que otros.
4. Votación o promedio final	▪ En clasificación, cada árbol “vota” por una clase. ▪ En regresión, se calcula el promedio de los valores predichos. El resultado final refleja **la decisión colectiva del bosque**.

Ejemplo:

Imagina que quieres predecir si **un cliente renovará su póliza**.
En lugar de preguntarle a un solo experto (un árbol), consultas a **100 expertos** (100 árboles),cada uno con distinta información del cliente y con su propia opinión.

Luego, tomas **la mayoría de votos**: Si 80 dicen “sí” y 20 “no”, el bosque predice que **el cliente renovará**.

Esa combinación de múltiples opiniones **reduce los errores individuales**.

Ventajas principales

1. **Alta precisión:**
   Al combinar muchos árboles, se reduce el error y se mejora la exactitud del modelo.
2. **Menor sobreajuste:**
   La aleatoriedad en los datos y variables evita que el modelo memorice los datos de entrenamiento.
3. **Evalúa importancia de variables:**
   Permite conocer qué características influyen más en la predicción.
4. **Versátil:**
   Funciona tanto para **clasificación** como para **regresión**.
5. **Maneja bien datos con muchas variables.**
6. **Tolera valores faltantes y datos ruidosos.**
7. **No requiere escalamiento de datos.**

Desventajas

1. **Menor interpretabilidad:**
   Aunque cada árbol individual es fácil de entender, el conjunto de cientos de árboles no lo es tanto.

2. **Mayor tiempo de cómputo:**
   Entrenar muchos árboles puede ser más lento, especialmente con grandes volúmenes de datos.

3. **Mayor consumo de memoria:**
   Necesita almacenar múltiples modelos.

Caso 3: Random Forest Predicción de supervivencia con el dataset titanic

Contexto:
La administración de una línea de cruceros quiere construir un modelo predictivo para identificar qué tipo de pasajeros tienen mayor probabilidad de sobrevivir a un naufragio, con el objetivo de optimizar la distribución de recursos de seguridad.

Objetivo:
Construir un modelo de clasificación binaria con random forest para predecir la variable survived.

Paso 1. Cargar y preparar los datos

```python
# Paso 1. Cargar y preparar el dataset+
# Importar librerías necesarias
import pandas as pd
import seaborn as sns
from sklearn.model_selection import train_test_split


# Cargar el dataset Titanic desde seaborn
titanic = sns.load_dataset("titanic")


# Seleccionar variables relevantes
cols = ['survived', 'pclass', 'sex', 'age', 'sibsp',
'parch', 'fare', 'embarked']
df = titanic[cols].dropna()  # eliminar valores nulos


# Convertir variables categóricas a numéricas mediante
One-Hot Encoding
df = pd.get_dummies(df, columns=['sex', 'embarked'],
drop_first=True)


# Mostrar las primeras filas
print(df.head())
```

```
   survived  pclass   age  sibsp  parch     fare  sex_male  embarked_Q  \
0         0       3  22.0      1      0   7.2500      True       False
1         1       1  38.0      1      0  71.2833     False       False
2         1       3  26.0      0      0   7.9250     False       False
3         1       1  35.0      1      0  53.1000     False       False
4         0       3  35.0      0      0   8.0500      True       False

   embarked_S
0        True
1       False
2        True
3        True
4        True
```

Explicación:

- Cargamos el dataset con `seaborn.load_dataset("titanic")`.
- Seleccionamos solo las columnas relevantes para evitar ruido.
- Usamos `dropna()` para eliminar filas con valores nulos.
- Las variables categóricas `sex` y `embarked` se codifican como variables dummy (`sex_male`, `embarked_Q`, `embarked_S`) para que el modelo las procese.

Paso 2. Paso 2. Separar variables predictoras y etiqueta

```python
#Paso 2. Separar variables
predictoras y etiqueta
# Variable objetivo (target)
y = df['survived']

# Variables predictoras (features)
X = df.drop(columns='survived')

# Mostrar dimensiones
print("Tamaño de X:", X.shape)
print("Tamaño de y:", y.shape)
```

```
Tamaño de X: (712, 8)
Tamaño de y: (712,)
```

Clave del proceso:

1. `X` contiene las características de cada pasajero: clase, edad, tarifa, género, etc.
2. `y` contiene la etiqueta binaria que el modelo aprenderá a predecir.

Esta separación es esencial para aplicar **aprendizaje supervisado**.

Paso 3. Paso 3. Dividir en entrenamiento y prueba

```python
# Paso 3. Dividir en entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

print("Datos de entrenamiento:", X_train.shape)
print("Datos de prueba:", X_test.shape)
```

```
Datos de entrenamiento: (498, 8)
Datos de prueba: (214, 8)
```

Explicación:

- 70 % de los datos se usa para **entrenar el modelo** y 30 % para **probarlo**.
- `random_state=42` garantiza reproducibilidad.

Paso 4. Entrenar el modelo de Random Forest

```python
# Paso 4. Entrenar el modelo de Random Forest
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Crear el modelo Random Forest
modelo_rf = RandomForestClassifier(
    n_estimators=100,     # número de árboles
    max_depth=5,          # profundidad máxima
    random_state=42
)

# Entrenar el modelo
modelo_rf.fit(X_train, y_train)

# Realizar predicciones
y_pred = modelo_rf.predict(X_test)

# Evaluar desempeño
print("Precisión del modelo (accuracy):", accuracy_score(y_test, y_pred))
print("\nReporte de clasificación:\n", classification_report(y_test, y_pred))
print("\nMatriz de confusión:\n", confusion_matrix(y_test, y_pred))
```

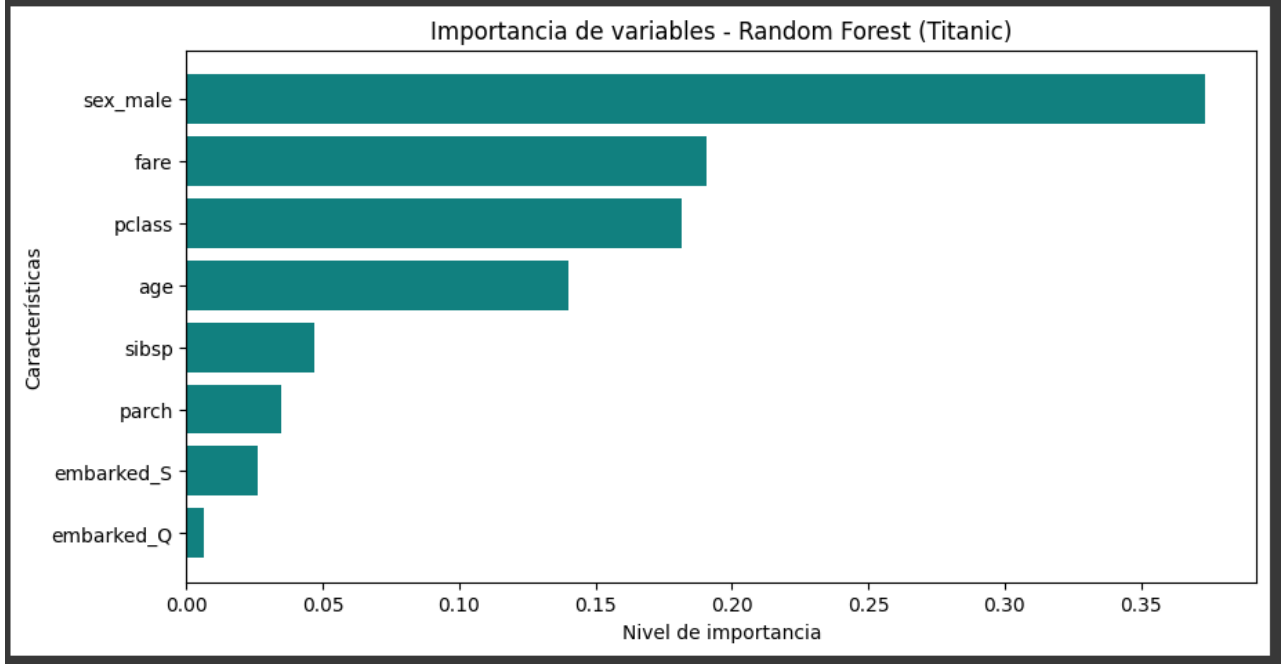
```
Precisión del modelo (accuracy): 0.780373831775701

Reporte de clasificación:
               precision    recall  f1-score   support

           0       0.77      0.87      0.82       122
           1       0.79      0.66      0.72        92

    accuracy                           0.78       214
   macro avg       0.78      0.77      0.77       214
weighted avg       0.78      0.78      0.78       214


Matriz de confusión:
 [[106  16]
 [ 31  61]]
```

Explicación docente:

- El modelo crea **100 árboles de decisión independientes** y combina sus resultados por **votación mayoritaria**.
- `max_depth=5` limita la complejidad de cada árbol, evitando sobreajuste.
- Calculamos métricas clave: **Accuracy**, **Precision**, **Recall**, **F1-score** y **Matriz de confusión**.

Paso 5. Importancia de las variables

```python
# Paso 5. Importancia de las variables
import matplotlib.pyplot as plt
import numpy as np

# Obtener importancia de características
importancias = modelo_rf.feature_importances_
indices = np.argsort(importancias)

plt.figure(figsize=(10,5))
plt.barh(range(len(indices)), importancias[indices], align='center', color='teal')
plt.yticks(range(len(indices)), [X.columns[i] for i in indices])
plt.title("Importancia de variables - Random Forest (Titanic)")
plt.xlabel("Nivel de importancia")
plt.ylabel("Características")
plt.show()
```



Interpretación visual:

- El gráfico muestra **cuánto aporta cada variable** a las decisiones del modelo.
- Cuanto más largo el bloque, mayor su importancia.

Paso 5. Importancia de las variables

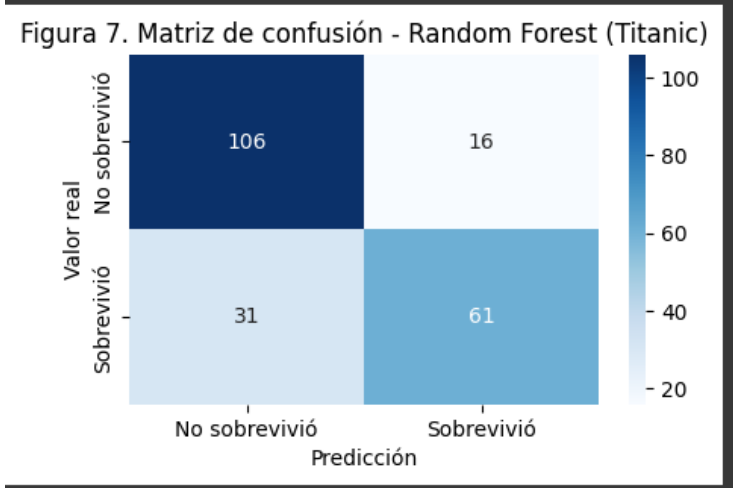
Variable	Importancia aproximada	Interpretación
sex_male	~0.37	La variable más influyente. Las mujeres tuvieron mayor tasa de supervivencia.
fare	~0.19	Pasajeros que pagaron tarifas más altas (clases altas) tuvieron más probabilidades de sobrevivir.
pclass	~0.18	La clase del pasaje influye: primera clase sobrevivió más.
age	~0.14	Edad relevante; los niños fueron priorizados.
embarked_Q, embarked_S, parch, sibsp	<0.05	Poco impacto individual, aunque aportan contexto familiar o geográfico.

Paso 6. Métricas de Evaluación

Métrica	Qué mide	Interpretación
Accuracy (Exactitud)	Proporción de predicciones correctas.	Mide desempeño global del modelo.
Precision (Precisión)	De los pasajeros predichos como “sobrevivientes”, cuántos lo fueron realmente.	Evalúa exactitud de los positivos.
Recall (Sensibilidad)	De los que sí sobrevivieron, cuántos fueron identificados correctamente.	Evalúa la capacidad del modelo para detectar sobrevivientes.
F1-Score	Media armónica entre Precisión y Recall.	Útil si hay clases desbalanceadas.
Matriz de confusión	Tabla con aciertos y errores.	Permite ver en qué clase se equivoca el modelo.

Paso 7. Evaluación de Desempeño

```python
# Paso 1. Matriz de confusión
# Librerías necesarias
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score

# Calcular la matriz de confusión
cm = confusion_matrix(y_test, y_pred)

# Visualización
plt.figure(figsize=(5,3))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['No sobrevivió', 'Sobrevivió'],
            yticklabels=['No sobrevivió', 'Sobrevivió'])
plt.title("Figura 7. Matriz de confusión - Random Forest (Titanic)")
plt.xlabel("Predicción")
plt.ylabel("Valor real")
plt.show()

# Calcular accuracy global
acc = accuracy_score(y_test, y_pred)
print(f"Exactitud global del modelo: {acc:.2%}")
```



Tipo	Significado
TP (Verdaderos Positivos)	Predichos como sobrevivientes y realmente sobrevivieron.
TN (Verdaderos Negativos)	Predichos como no sobrevivientes y realmente no sobrevivieron.
FP (Falsos Positivos)	Predichos como sobrevivientes, pero no lo eran.
FN (Falsos Negativos)	Predichos como no sobrevivientes, pero sí lo eran.

Interpretaciones clave:

- **Exactitud global (Accuracy):** porcentaje de aciertos totales.
- **Recall (Sensibilidad):** capacidad del modelo para detectar sobrevivientes.
- **Precision:** qué tan confiable es cuando predice que alguien sobrevivirá.

Paso 8. Reporte de métricas

```python
# Paso 2. Reporte de métricas
# Reporte detallado
print("Figura 8. Reporte de clasificación\n")
print(classification_report(y_test, y_pred, target_names=['No sobrevivió', 'Sobrevivió']))
```

```
Figura 8. Reporte de clasificación

              precision    recall  f1-score   support

No sobrevivió       0.77      0.87      0.82       122
   Sobrevivió       0.79      0.66      0.72        92

    accuracy                           0.78       214
   macro avg       0.78      0.77      0.77       214
weighted avg       0.78      0.78      0.78       214
```

Interpretación:

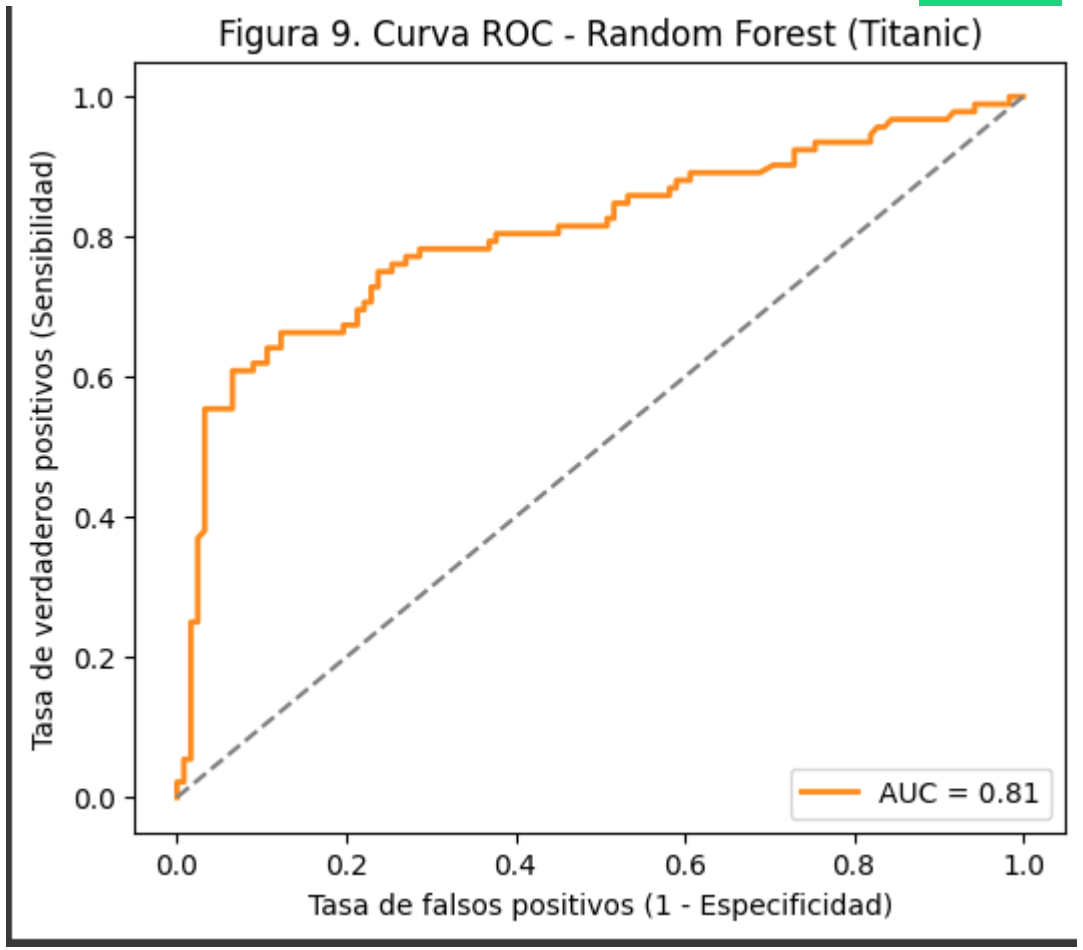
- **Precisión (Precision):**
  - No sobrevivió: 77.4 % de aciertos.
  - Sobrevivió: 78.7 % de aciertos.
- **Sensibilidad (Recall):**
  - Detecta 86.9 % de los fallecidos.
  - Detecta 66.3 % de los sobrevivientes (a mejorar).
- **F1-Score:** equilibrio entre precisión y recall.
  - Clase 0 (no sobrevivió): 0.82 (excelente).
  - Clase 1 (sobrevivió): 0.72 (aceptable).
- **Accuracy global:** ~78 %.

Interpretación para negocio:

1. El modelo es **muy bueno detectando a los pasajeros que no sobrevivirán**,
   pero tiene **margen de mejora al identificar sobrevivientes**.
2. Su desempeño balanceado (78 % macro avg) indica **buen equilibrio y sin sesgo marcado**.

Paso 9. Curva ROC y AUC

```python
# Paso 3. Curva ROC y AUC
from sklearn.metrics import roc_curve, auc

# Calcular probabilidades de predicción
y_prob = modelo_rf.predict_proba(X_test)[:, 1]

# Calcular curva ROC
fpr, tpr, thresholds = roc_curve(y_test, y_prob)
roc_auc = auc(fpr, tpr)

# Graficar curva ROC
plt.figure(figsize=(6,5))
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'AUC = {roc_auc:.2f}')
plt.plot([0, 1], [0, 1], color='gray', linestyle='--')
plt.xlabel('Tasa de falsos positivos (1 - Especificidad)')
plt.ylabel('Tasa de verdaderos positivos (Sensibilidad)')
plt.title('Figura 9. Curva ROC - Random Forest (Titanic)')
plt.legend(loc='lower right')
plt.show()
```



Paso 9. Curva ROC y AUC

Interpretación visual

•**Eje X:** tasa de falsos positivos (FPR).
•**Eje Y:** tasa de verdaderos positivos (TPR).
•**AUC (Área bajo la curva):** mide la capacidad del modelo para distinguir entre clases.

Resultados típicos:
•AUC ≈ **0.82** → 82 % de probabilidad de distinguir correctamente entre sobrevivientes y no sobrevivientes.
•Escala:

- 0.5 → modelo aleatorio
- 1.0 → modelo perfecto

Interpretación para toma de decisiones:

•El **punto óptimo** está cerca de la esquina superior izquierda (alta sensibilidad, baja tasa de error).
•Un **AUC de 0.82** indica un **modelo sólido y discriminativo**.
•En un contexto de seguridad marítima:

- Podemos **ajustar el umbral** para detectar más sobrevivientes, aunque aumenten falsos positivos.
- O **optimizar recursos** buscando equilibrio entre costos y seguridad.

Métrica	Valor aproximado	Interpretación
Accuracy	78 %	Desempeño global correcto.
Precision	78 %	Predicciones positivas razonablemente precisas.
Recall	66 % (sobrevivientes)	Detecta bien a quienes no sobreviven; mejora pendiente para positivos.
F1-score	0.75	Buen balance entre precisión y sensibilidad.
AUC	0.82	Excelente capacidad discriminativa.

Conclusión:
El modelo **Random Forest** ofrece una predicción **robusta, balanceada y confiable**, con **mejor desempeño que un árbol individual**.
Su interpretación es útil para **priorizar decisiones de seguridad** o **analizar perfiles de riesgo**.

Evaluación adicional: Modelos de Regresión con Random Forest

Para casos **numéricos**, usamos **RandomForestRegressor**.

Ejemplo aplicado al dataset **Diabetes** de `sklearn`:

```python
# Código de evaluación de regresión
from sklearn.datasets import load_diabetes
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

# Cargar datos
diabetes = load_diabetes(as_frame=True)
X, y = diabetes.data, diabetes.target

# División entrenamiento/prueba
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Entrenar modelo
modelo_reg = RandomForestRegressor(n_estimators=100, max_depth=5, random_state=42)
modelo_reg.fit(X_train, y_train)

# Predicciones
y_pred = modelo_reg.predict(X_test)

# Métricas
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

print("MAE:", round(mae, 2))
print("RMSE:", round(rmse, 2))
print("R²:", round(r2, 2))
```

```
MAE: 42.18
RMSE: 52.62
R²: 0.49
```

Métrica	Valor	Interpretación
MAE	42.18	En promedio, el modelo se equivoca 42 unidades al predecir.
RMSE	52.62	Penaliza errores grandes; existen algunas predicciones alejadas.
R^2	0.49	El modelo explica el 49 % de la variabilidad de la progresión diabética.

Conclusión del modelo de regresión:

1. Desempeño **aceptable**, pero con **espacio de mejora**.
2. Un R^2 cercano a **0.7 o superior** sería deseable en contextos clínicos.
3. Posibles mejoras: aumentar profundidad, ingeniería de variables o usar más datos.

Para casos **numéricos**, usamos **RandomForestRegressor**.

Ejemplo aplicado al dataset **Diabetes** de `sklearn`:

Tipo de modelo	Uso principal	Métricas clave	Ejemplo
RandomForestClassifier	Clasificación (binaria o múltiple)	Accuracy, Precision, Recall, AUC	Titanic (sobrevivencia)
RandomForestRegressor	Regresión (valores numéricos)	MAE, RMSE, R^2	Diabetes (progresión de enfermedad)

"Evaluar un modelo no es solo medir su precisión, sino entender **dónde se equivoca, por qué y cómo mejorarlo**.
Las métricas no son números: Son una brújula que orienta nuestras decisiones técnicas y de negocio."

Receso

07:00

CONNECT

TECMILENIO

Aprendizaje Automático y modelado predictivo

Tema 6: Aplicación en Python del modelo K vecinos más cercanos

Mtra. Fabiola Cristina Olguín Barrón

TECMILENIO

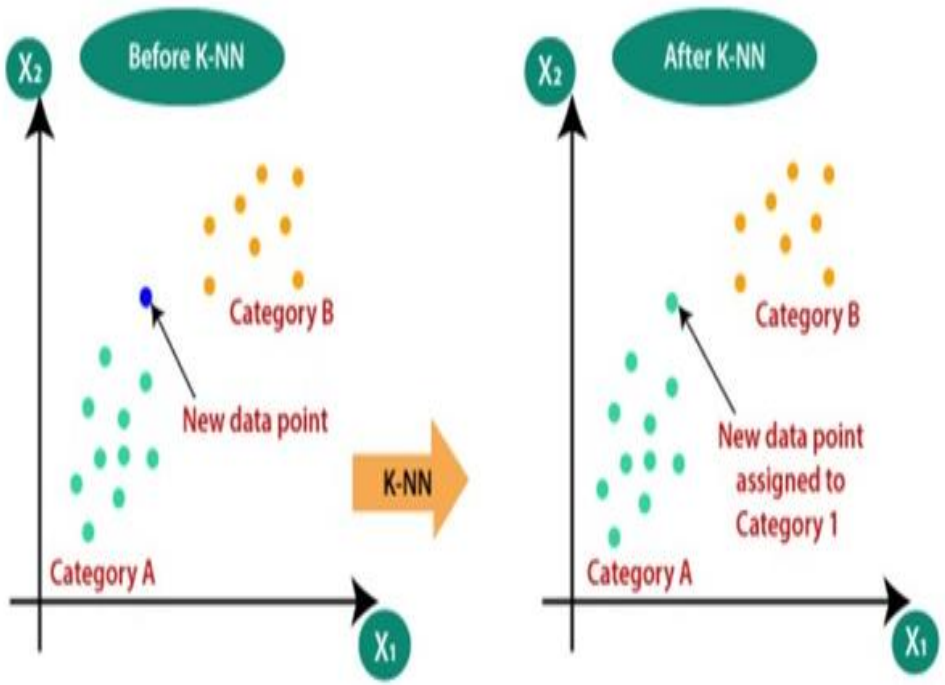
¿Qué es K vecinos mas cercanos?

El algoritmo **K-Nearest Neighbors (KNN)** es un método de **aprendizaje supervisado** usado tanto para **clasificación** como para **regresión**.

Su idea principal es **predecir el valor o clase de un punto nuevo** observando los **K puntos más cercanos** en el conjunto de entrenamiento.

Imagina que tienes muchos puntos en un plano (datos conocidos) y llega un nuevo punto (dato desconocido).

KNN **busca los K puntos más cercanos para realizar la determinación si pertenece a una categoría o a otra.**



Funcionamiento General del Modelo

1. Selecciona el número de vecinos K (Parámetro K).
2. Calcula la distancia entre el nuevo punto y todos los puntos del conjunto de entrenamiento.
3. Identifica los K puntos más cercanos.
4. Determina la clasificación del nuevo punto o valor:
   - **En clasificación:** Asigna la clase más frecuente.
   - **En regresión:** Promedia los valores de los vecinos.

Parámetro clave: K

- **K** es el número de vecinos que se consideran.
- Un **K pequeño (p. ej. K=1)** hace al modelo muy sensible al ruido (sobreajuste).
- Un **K muy grande** suaviza demasiado las predicciones (subajuste).
- En la práctica, se elige K probando varios valores y eligiendo el que dé mejor desempeño en validación cruzada.

Distancia: Se puede determinar a través de:

- **La distancia Euclidiana (La distancia que existe entre dos puntos).**
- La distancia Manhattan (Suma las **diferencias absolutas** entre las coordenadas de dos puntos).
- **Distancia Minkowski (L^p)**
  Es una **versión general** que engloba las dos anteriores.

Clasificación de distancias

1. Distancia Euclidiana

Es la distancia **más común** y representa la "línea recta" entre dos puntos en un espacio n-dimensional.
Matemáticamente se expresa como:

$$d(p,q) = \sqrt{\sum_{i=1}^{n}(x_i - y_i)^2}$$

Se usa cuando las variables están **en la misma escala** (por ejemplo, edad, altura, peso).
Es ideal para datos **continuos** donde la relación entre variables es **geométrica o física**.

Ejemplo práctico:

En un modelo que predice si un cliente comprará un producto con base en su **edad** y **salario**, la distancia euclidiana medirá qué tan "lejos" está un cliente nuevo de los perfiles que ya han comprado.

2. Distancia Manhattan

Esta distancia suma las **diferencias absolutas** entre las coordenadas de dos puntos.
Su fórmula es:

$$d(p, q) = \sum_{i=1}^{n} | x_i - y_i |$$

Se conoce también como "**distancia de ciudad o de taxi**", porque representa el recorrido si te movieras solo en líneas rectas (como por avenidas y calles en una cuadrícula).
Es útil cuando los datos tienen **valores atípicos (outliers)** o **diferente escala** entre variables, porque penaliza menos los grandes saltos que la Euclidiana.

Ejemplo práctico:

Clasificar zonas de la ciudad según su nivel de tráfico o contaminación, donde las variables (ruido, tránsito, densidad) tienen escalas diferentes.

3. Distancia Minkowski

Es una **versión general** que engloba las dos anteriores.
Su fórmula es:

$$d(p, q) = \left(\sum_{i=1}^{n} | x_i - y_i |^p\right)^{1/p}$$

- Si $p = 1$, equivale a la **distancia Manhattan**.
- Si $p = 2$, equivale a la **distancia Euclidiana**.

Permite ajustar el parámetro p para **controlar la sensibilidad** a las diferencias entre las variables.
Cuando $p$es alto, las diferencias grandes pesan más.

Ejemplo práctico:

En reconocimiento de imágenes, ajustar p puede hacer que el modelo sea más sensible o menos a ciertas características visuales (por ejemplo, color o textura dominante).

Clasificación de distancias

Distancia	Idea principal	Cuándo usarla	Ejemplos reales de aplicación
Euclidiana	Mide la distancia **en línea recta**, como si trazáramos una cuerda entre dos puntos. Representa el camino más corto entre ellos.	▪ Cuando las variables están **en la misma escala** (por ejemplo, cm, kg, años). ▪ Cuando **no hay valores atípicos**. ▪ En datos **continuos o geométricos**.	▪ **Reconocimiento facial y de voz:** Compara vectores de características. ▪ **Robótica y navegación:** Calcula trayectos directos entre posiciones. ▪ **GPS y geolocalización:** Mide la distancia directa entre coordenadas.
Manhattan	Mide la distancia **por ejes**, como si caminaras por calles en forma de cuadrícula (solo avanzas en líneas rectas).	▪ Cuando hay **valores atípicos o ruido**. ▪ Cuando las variables tienen **diferentes escalas**. ▪ En datos **discretos o con saltos numéricos**, donde el movimiento es por pasos o bloques.	▪ **Sistemas de transporte urbano (Uber, Waze):** Distancias entre calles. ▪ **Recomendadores (Netflix, Spotify):** Comparación de perfiles de usuario por componentes. ▪ **Análisis clínico:** Datos con valores muy dispersos o categorizados.
Minkowski	Es una distancia **flexible o generalizada** que puede comportarse como Euclidiana o Manhattan, según el parámetro que se ajuste.	▪ Cuando se desea **ajustar la sensibilidad** del modelo a diferencias grandes o pequeñas. ▪ Cuando **no se sabe cuál métrica** es mejor y se busca optimizar. ▪ En datos **mixtos o multivariados**.	• **Machine Learning (KNN, K-Means, DBSCAN):** Permite probar distintos valores de sensibilidad. • **Análisis financiero:** ajusta la penalización por diferencias grandes en precios o ingresos. • **Bioinformática:** compara expresiones genéticas o secuencias de ADN con distintas ponderaciones.

Métricas de evaluación del Modelo KNN

Métrica	Qué mide	Interpretación	Ejemplo aplicado al modelo Iris
Matriz de confusión	Compara las predicciones del modelo con los valores reales.	Muestra **en qué clases acierta o se equivoca** el modelo.	Permite ver si el modelo confunde, por ejemplo, flores versicolor con virginica. Si una clase tiene muchos errores, indica que el modelo no la distingue bien.
Precisión global (Accuracy)	Porcentaje total de aciertos del modelo.	Evalúa el **desempeño global**, sin distinguir por clase.	Si el KNN logra un accuracy del 96 %, significa que acierta en 96 de cada 100 flores.
Precisión por clase (Precision)	Qué tan confiables son las predicciones positivas de una clase.	De todas las veces que el modelo dijo “es setosa”, mide **cuántas veces acertó**.	Si la precisión de virginica es 0.90, 9 de cada 10 veces que el modelo predijo “virginica”, fue correcto.
Sensibilidad (Recall)	Cuántos casos reales de una clase fueron detectados correctamente.	Mide la capacidad del modelo para **no dejar escapar casos reales**.	Si el recall de setosa es 1.0, el modelo detectó correctamente todas las setosa del conjunto de prueba.
F1-Score	Equilibrio entre precisión y recall.	Resume el rendimiento de cada clase en una sola métrica; útil cuando hay **clases desbalanceadas**.	Si virginica tiene precisión alta (0.9) pero recall bajo (0.7), su F1 ≈ 0.8, mostrando el compromiso entre ambas.
Curva ROC / AUC (solo en binario)	Mide la capacidad del modelo para separar clases.	Cuanto más cerca esté el AUC de 1, **mejor distingue** entre clases positivas y negativas.	Si se convierte el problema a “Setosa vs No Setosa”, un AUC de 0.98 indica una excelente separación entre ambas clases.

¿Qué es la Matriz de Confusión?

La **Matriz de Confusión** sirve para saber **qué tan bien predice el modelo**.

La **Matriz de Confusión** nos ayuda a ver **no solo cuántas veces acierta**, sino también **en qué se equivoca**.

Estructura de la Matriz de Confusión:

Cuando el modelo clasifica más de dos clases, como en el caso del "*Iris*", donde tenemos tres tipos de flores— la matriz de confusión se amplía.

Cada fila representa las **clases reales**, y cada columna las **predicciones del modelo**

Clase real / Predicha	Setosa Predicha	Versicolor Predicha	Virginica Predicha
Setosa (Real)	Acertó (TP para Setosa)	Confundida con Versicolor (Valor <> 0 son errores)	Confundida con Virginica (Valor <> 0 son errores)
Versicolor (Real)	Confundida con Setosa (Valor <> 0 son errores)	Acertó (TP para Versicolor)	Confundida con Virginica (Valor <> 0 son errores)
Virginica (Real)	Confundida con Setosa (Valor <> 0 son errores)	Confundida con Versicolor (Valor <> 0 son errores)	Acertó (TP para Virginica)

Verde = Aciertos de cada clase

Rojo = Errores de predicción (En este caso flores que el modelo confundió con otra especie)

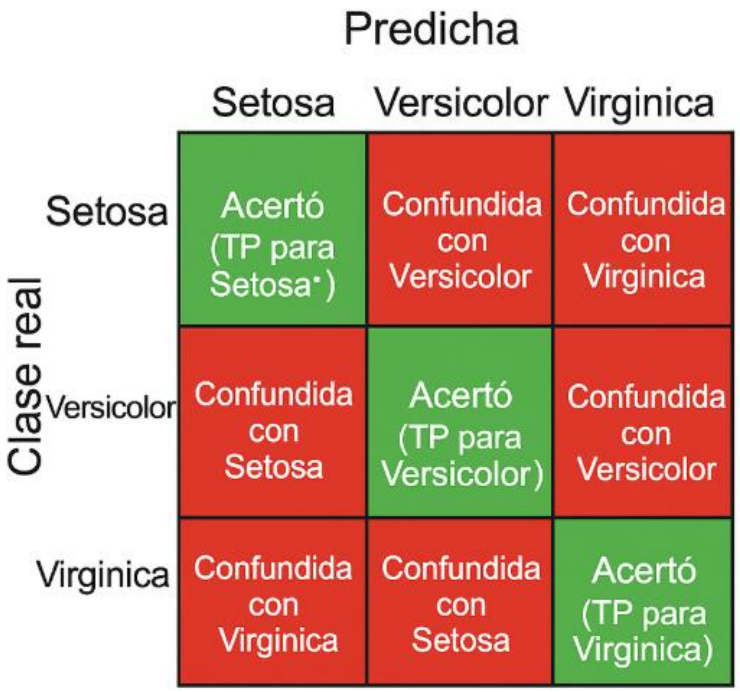
Matriz de Confusión Real (IRIS):

Interpretación rápida:

- La **diagonal principal** (↘) muestra los **aciertos** del modelo.
- Los valores **fuera de la diagonal** son los **errores o confusiones**.
- Cada fila te dice **qué tanto el modelo acierta o falla** para una clase específica.

Clase real / Predicha	Setosa	Versicolor	Virginica
Setosa (Real)	10	0	0
Versicolor (Real)	0	9	1
Virginica (Real)	0	1	9

- Estas métricas permiten **evaluar el rendimiento global y por clase**, y detectar si alguna categoría se está **clasificando sistemáticamente mal**.
  Con ellas puedes decidir si el modelo KNN necesita ajustar el valor de *K* o escalar mejor las variables.



¿Qué es la Curva ROC?

La **Curva ROC** (Receiver Operating Characteristic) nos muestra **qué tan bien un modelo distingue entre dos clases**:
Por ejemplo, entre *Sí / No*, *Fraude / No fraude*, o *Enfermo / Sano*.
En el caso del **modelo KNN**, solo podemos graficar esta curva si el problema tiene **dos clases** (binario).
En el dataset *Iris*, podríamos simplificarlo así:

- Clase positiva: *Setosa*
- Clase negativa: *No Setosa* (que agrupa *Versicolor* y *Virginica*).

Qué representa la curva:

La curva ROC compara dos cosas:

- **Eje X → Falsos Positivos (1 - Especificidad)**
  Qué tanto el modelo **se equivoca marcando como positivos casos que no lo son**.
- **Eje Y → Verdaderos Positivos (Sensibilidad o Recall)**
  Qué tanto el modelo **acierta detectando casos positivos**.

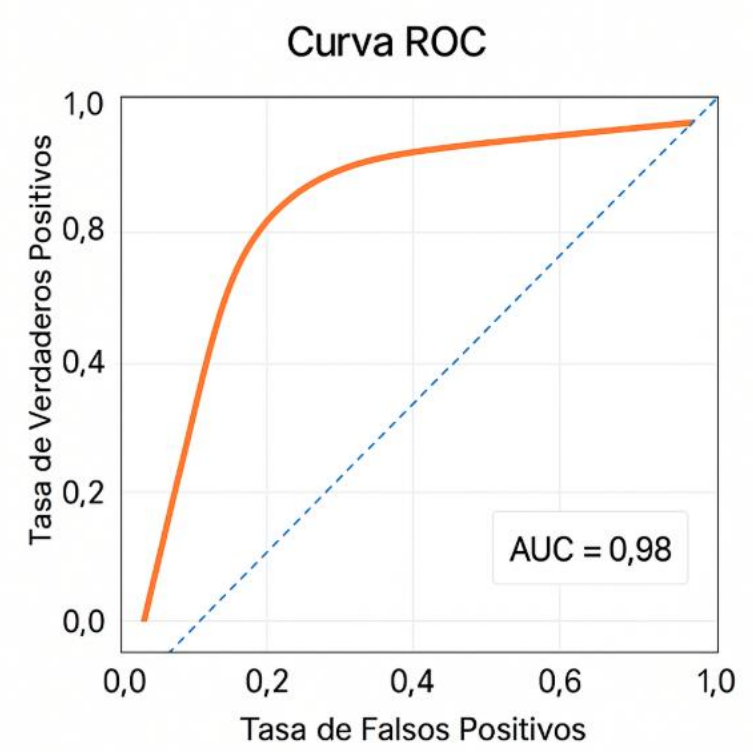
¿Qué representa la curva?

La curva ROC compara dos cosas:

1. **Eje X → Falsos Positivos**
   Qué tanto el modelo **se equivoca marcando como positivos casos que no lo son**.
2. **Eje Y → Verdaderos Positivos**
   Qué tanto el modelo **acierta detectando casos positivos**.

¿Cómo se interpreta visualmente?

Forma de la curva	Interpretación
Curva alta y hacia la esquina superior izquierda	Excelente modelo: Detecta bien los positivos y evita los falsos.
Curva diagonal (45°)	Modelo aleatorio: No distingue entre clases (como adivinar).
Curva por debajo de la diagonal	Modelo invertido (predice al revés).



Resumen del Modelo KNN

Aspecto	Descripción
Tipo de aprendizaje	Supervisado (requiere datos etiquetados).
Tipo de problema	Clasificación o Regresión.
Idea principal	Un nuevo dato se clasifica o predice **observando las K instancias más cercanas** en el conjunto de entrenamiento.
Funcionamiento básico	a) Se calcula la distancia entre el nuevo punto y todos los puntos del conjunto de datos. b) Se eligen los **K vecinos más cercanos**. c) En clasificación: se asigna la clase **más frecuente**. d) En regresión: se toma el **promedio** de los valores vecinos.
Parámetro clave	**K (número de vecinos):** Valores pequeños → modelo sensible (puede sobreajustar). Valores grandes → modelo más general (puede subajustar).
Métricas de distancia	**Euclidiana:** línea recta entre puntos. **Manhattan:** distancia por ejes (tipo "cuadras"). **Minkowski:** generaliza las anteriores según el parámetro (p).
Requisitos importantes	Escalar o normalizar las variables (KNN es sensible a la magnitud de las escalas). Elegir un K adecuado mediante validación cruzada.
Ventajas	▪ Sencillo de entender e implementar. ▪ No requiere entrenamiento complejo. ▪ Adaptable a diferentes tipos de datos.
Desventajas	Costo alto en predicción (debe calcular distancias a todos los puntos). Sensible a variables irrelevantes o ruido. Requiere escalado de datos. Difícil de interpretar con datos de alta dimensión.

Resumen del Modelo KNN

Aspecto	Descripción
Métricas de evaluación (clasificación)	❑ Exactitud (Accuracy) ❑ Precisión (Precision) ❑ Recall (Sensibilidad) ❑ F1-Score- Matriz de confusión
Métricas de evaluación (regresión)	❑ MAE (Error absoluto medio) ❑ MSE (Error cuadrático medio) ❑ RMSE (Raíz del error cuadrático medio) ❑ (R^2) (Coeficiente de determinación)
Usos comunes	❑ Clasificación de clientes o segmentos de mercado. ❑ Diagnóstico médico (ej. Tipo de tumor). ❑ Clasificación de especies biológicas (Iris dataset). ❑ Sistemas de recomendación o reconocimiento de patrones.
Implementación típica (Python)	from sklearn.neighbors import KNeighborsClassifier model = KNeighborsClassifier(n_neighbors=5) model.fit(X_train, y_train) y_pred = model.predict(X_test)

Tema 4

Vamos a Colab a ver como se ejecuta el código del tema 6.

Receso

CONNECT

TECMILENIO

Aprendizaje Automático y modelado predictivo

Desarrollo de la evidencia de la semana

TECMILENIO

1. Actividad a desarrollar en la semana Evidencia 1

Etapa / Sección	Qué debes hacer	Detalles o ejemplos	Entregable en PDF o Colab
1. Selección del dataset	Escoge un conjunto de datos con sentido para ti (profesional, académico o público).	Debe tener **≥200 registros**, **1 variable objetivo** (numérica o categórica) y **≥4 variables predictoras**. Si es confidencial, **anonimiza** los datos.	Explicación breve en el PDF.
2. Definición del problema	Formula una **pregunta clara** de análisis.	Ejemplo regresión: ¿Cuál será el costo del seguro según el BMI y edad? Ejemplo clasificación: ¿Sobrevivirá el pasajero según su sexo y clase?	Explicación en el PDF.
3. Contexto del caso	Describe el origen y propósito de los datos.	Indica de dónde vienen, qué quieres resolver y por qué es útil.	Texto introductorio en el PDF.
4. Preparación de los datos	Limpia y transforma la información.	Carga del dataset, eliminación de nulos, codificación de variables categóricas, normalización si aplica.	Código y comentarios en Colab + resumen en PDF.
5. Selección del modelo ML	Aplica un modelo supervisado según tu problema.	Opciones: Regresión lineal, polinomial, logística, Árbol de decisión, Random Forest o KNN.	Código del modelo en Colab.
6. Entrenamiento y evaluación	Entrena el modelo y calcula las métricas adecuadas.	**Regresión:** R^2, MAE, MSE, RMSE. **Clasificación:** Accuracy, Precision, Recall, F1, Matriz de confusión.	Resultados y métricas en Colab + interpretación en PDF.
7. Visualización de resultados	Muestra tus resultados gráficamente.	Usa gráficos de dispersión, barras o matriz de confusión.	Gráficas en Colab + captura o resumen en PDF.
8. Interpretación y conclusión	Analiza los hallazgos y aporta valor.	Resume qué aprendiste y qué utilidad tiene el modelo en la práctica.	Conclusión ejecutiva en PDF.
9. Revisión final y entrega	Verifica que todo esté completo y ordenado.	Revisa portadas, comentarios y formato profesional.	**2 archivos finales:** Google Colab (.ipynb) Reporte en PDF

Recomendación para tu Evidencia 1

Scikit-learn (sklearn.datasets) trae varios datasets listos para practicar *aprendizaje supervisado*, y **sí puedes usar varios de ellos para tu Evidencia 1**, cumpliendo con el requisito de tener más de **200 registros** y una **variable objetivo clara**.

- Si vas a trabajar **clasificación**, usa `load_breast_cancer()` (muy completo y con buen tamaño).
  Clasifica tumores como malignos o benignos con base en características celulares.

- Si prefieres **regresión**, usa `fetch_california_housing()` (caso real, datos amplios y métricas claras).
  Predice el valor promedio de viviendas en California con base en características socioeconómicas.

```
from sklearn.datasets import
load_breast_cancer
import pandas as pd

# Cargar dataset
data = load_breast_cancer()

# Convertir a DataFrame
df = pd.DataFrame(data.data,
columns=data.feature_names)
df['target'] = data.target

# Vista general
df.head()

# Variable objetivo (target): 0 = maligno,
1 = benigno
# Tipo de problema: Clasificación binaria
```

```
from sklearn.datasets import fetch_california_housing
import pandas as pd

# Cargar dataset
data = fetch_california_housing()

# Convertir a DataFrame
df = pd.DataFrame(data.data,
columns=data.feature_names)
df['target'] = data.target

# Vista general
df.head()

# Variable objetivo (target): target = valor promedio
de la vivienda
# Tipo de problema: Regresión
```

Explicación a 1 variable objetivo (numérica o categórica) y ≥4 variables predictoras.
Si es confidencial, anonimiza los datos.

Elemento	Qué significa	Ejemplo práctico
≥200 registros	El dataset debe tener **al menos 200 filas** o casos observados. Es decir, 200 ejemplos distintos (personas, productos, viviendas, etc.).	Un archivo con 500 pacientes o 250 viviendas cumple el requisito.
1 variable objetivo (target)	Es la **columna que se quiere predecir**. Puede ser: ❑ Numérica → Si haces **regresión** (por ejemplo, precio, edad, costo). ❑ Categórica → Si haces **clasificación** (por ejemplo, “sí/no”, “tipo A/B/C”).	En el dataset de vivienda: target = precio. En el de cáncer: target = tipo de tumor.
≥4 variables predictoras	Son las **características** o **columnas** que usarás para hacer la predicción. Deben ser al menos cuatro.	En el caso de vivienda: IngresoMedio, habitaciones, poblacion, latitud.
Anonimizar los datos	Si usas datos de tu trabajo o empresa, debes **eliminar o cambiar nombres, correos, identificadores** o cualquier información personal o sensible.	Cambiar “Juan Pérez” por “Cliente 001” o quitar columnas como CURP o teléfono.

Preguntas de la actividad

¡Gracias por su atención!

TECMILENIO